

Full Stack Development with MERN

Project Documentation

1. Introduction

Project Title: Online Complaint Registration and Management System

Team Members: N. Sri Gowri Naga Hymavathi — Full Stack Developer (Frontend, Backend, Database, Testing)

2. Project Description

The Online Complaint Registration and Management System is a full-stack web application built using the MERN stack (MongoDB, Express.js, React.js, Node.js). It provides a digital platform for citizens to register civic complaints such as water leakage, power outages, and road damage online, eliminating the need for physical office visits or phone calls. The system supports three user roles — ordinary users who file and track complaints, agents who are assigned to resolve them, and admins who oversee the entire platform. Users can submit complaints with image evidence, track real-time status updates, and communicate directly with assigned agents through a built-in chat window. The backend follows the MVC (Model-View-Controller) architectural pattern with JWT-based authentication for secure role-based access control. MongoDB stores all user profiles, complaints, and messages efficiently, while the React.js frontend provides a responsive and intuitive interface using Bootstrap and Material UI.

3. Scenario

Ravi is a citizen who notices a water leakage issue on his street. Instead of visiting the municipal office, he visits the Online Complaint Portal, registers an account, and logs in. He browses departments, selects Municipal Corporation and Water Leakage category, fills in the complaint details, uploads a photo of the leak, and submits the complaint. He immediately receives a tracking number. An admin reviews the complaint on the dashboard and assigns it to a field agent. Ravi receives a notification that his complaint is Assigned. The agent logs into the system, views the complaint, updates the status to In Progress, and chats with Ravi for additional details. Once the technician fixes the leak, the agent marks the complaint as Resolved and uploads proof. Ravi can see the complete status timeline on his dashboard.

4. System Requirements

4.1 Software Requirements

- Operating System: Windows 10/11, macOS, or Linux
- Node.js (v18 or above): Runtime environment for frontend and backend
- npm (v8 or above): Package manager for installing dependencies
- React.js: JavaScript library for building dynamic user interfaces
- Express.js: Lightweight web framework for building RESTful APIs

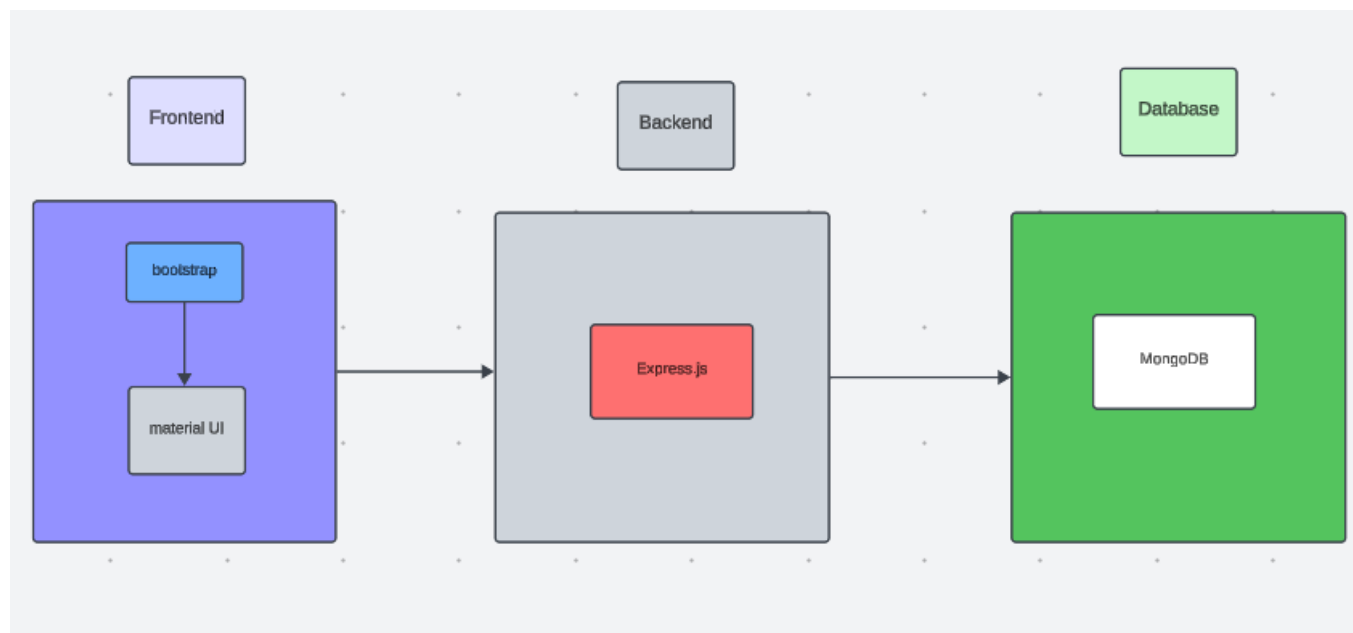
- MongoDB: NoSQL database for storing users, complaints and messages
- Mongoose: ODM library for MongoDB schema modeling
- Postman: Tool for testing and validating API endpoints
- Visual Studio Code: Preferred code editor with terminal support
- MongoDB Compass: GUI tool for managing MongoDB database
- Git: Version control system for tracking code changes

4.2 Hardware Requirements

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better
- RAM: Minimum 8 GB (16 GB recommended) for running development servers simultaneously
- Storage: At least 1 GB free space for packages, MongoDB setup, and project files
- Display: 1366x768 or higher for optimal coding and application visualization

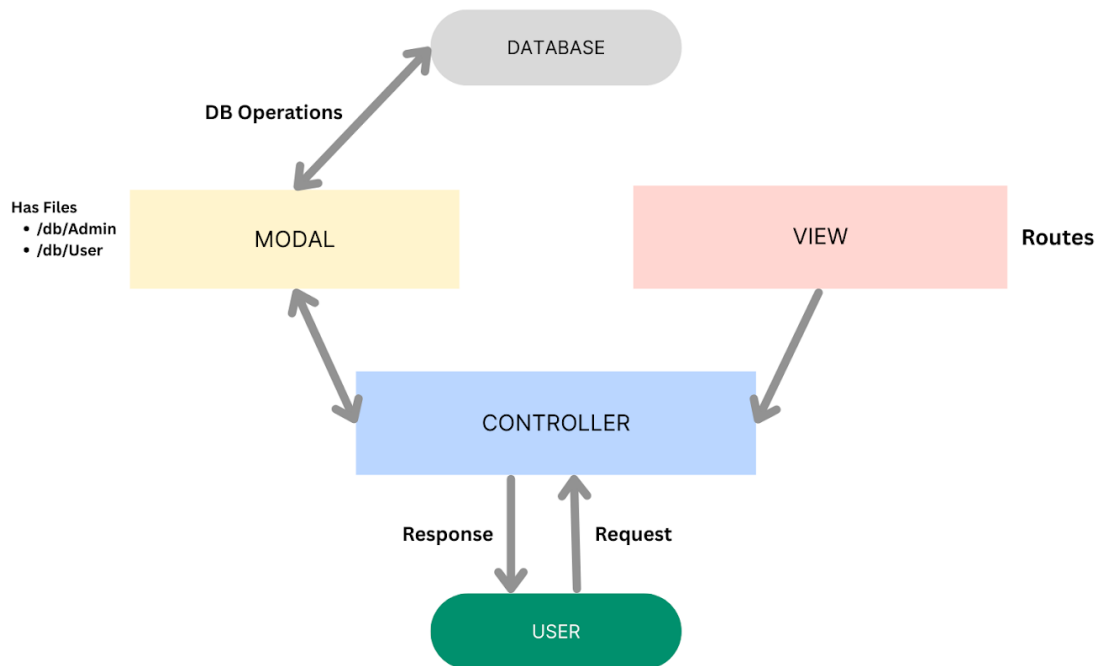
5. Project Architecture

5.1 Technical Architecture



The application follows a client-server model where the frontend acts as the client and the backend acts as the server. The frontend is built with React.js using component-based architecture, communicates with the backend through RESTful APIs using the Axios library, and uses Bootstrap and Material UI for a responsive user interface. The backend is built with Express.js following the MVC architectural pattern, handles authentication using JWT tokens, and uses Mongoose ODM to interact with MongoDB. The database stores three main collections: users, complaints, and messages.

5.2 MVC Pattern



The backend follows the Model-View-Controller (MVC) architectural pattern which separates the application into three interconnected layers:

Model Layer (Data Layer)

The Model layer handles all data-related logic including schema definition and database operations. Models are implemented using Mongoose which provides a schema-based solution for MongoDB. The project has three models: User.js (stores name, email, phone, password, role), Complaint.js (stores title, description, category, department, urgency, status, attachment, userId, agentId), and Message.js (stores complaintId, senderId, senderRole, text).

Controller Layer

The Controller layer acts as an intermediary between routes and models. It receives incoming requests, validates and processes input, calls appropriate model methods, and returns responses. The project has four controllers: authController.js (register, login), complaintController.js (create, get, update), adminController.js (dashboard, assign, manage), and messageController.js (get, send).

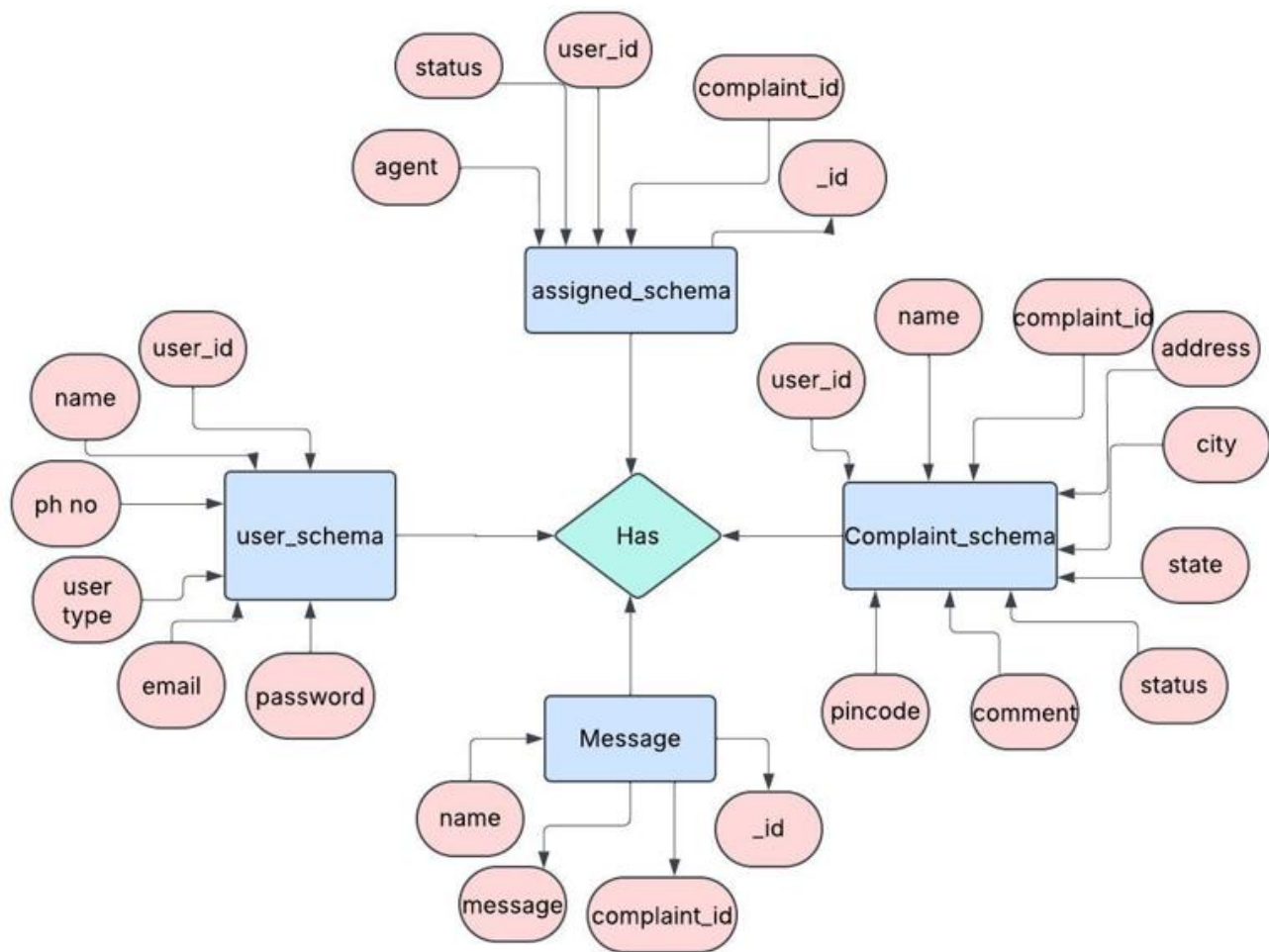
View Layer (Routing Layer)

The View layer is implemented as the routing layer where REST API endpoints are defined. These endpoints handle HTTP requests (GET, POST, PUT, DELETE) and invoke the appropriate controller functions. The project has four route files: authRoutes.js, complaintRoutes.js, adminRoutes.js, and messageRoutes.js.

Advantages of MVC:

- Separation of Concerns: Each layer has a clearly defined responsibility
- Scalability: New features can be added by creating new routes, controllers and models
- Reusability: Logic in controllers and models can be reused across the application
- Testability: Each layer can be tested independently
- Collaboration-Friendly: Multiple developers can work on different layers simultaneously

5.3 ER Diagram



The database has three collections with the following relationships:

- users — stores user profiles with fields: _id, name, email, phone, password (hashed), role (user/agent/admin), createdAt
- complaints — stores complaint data with fields: _id, title, description, category, department, urgency, status, attachment, userId (ref: users), userName, userEmail, agentId (ref: users), createdAt
- messages — stores chat messages with fields: _id, complaintId (ref: complaints), senderId (ref: users), senderRole, text, createdAt

Relationships: One user can file many complaints (1:N). One complaint can have many messages (1:N). One agent can be assigned many complaints (1:N).

5.4 Features

- User registration and login with JWT authentication
- Complaint submission form with title, category, department, urgency, description
- Image and evidence upload using Multer middleware
- Real-time complaint status tracking with visual timeline
- Role-based access control for User, Agent and Admin
- Admin dashboard with statistics — total, pending, in progress, resolved
- Admin complaint assignment to agents based on workload
- Agent status updates — Assigned, In Progress, Resolved, Rejected
- Built-in chat window between user and agent
- Profile management for users

5.5 Roles and Responsibilities

User (Citizen):

- Register and login to the platform
- Browse departments and complaint categories
- Submit complaints with image evidence
- Track complaint status in real time
- Communicate with assigned agent via chat

Agent (Officer):

- Login and view assigned complaints dashboard
- Update complaint status based on progress
- Communicate with users through chat window
- Resolve complaints and upload proof

Admin:

- Monitor all complaints on the dashboard
- Assign complaints to agents based on expertise
- Manage user and agent accounts
- Remove spam or misuse complaints
- Enforce platform policies and ensure data security

5.6 User Flow

1. User visits home page and clicks Register
2. User fills registration form (name, email, phone, password) and submits
3. User logs in and is redirected to role-based dashboard
4. User clicks New Complaint, fills form, uploads evidence, submits
5. Admin sees new complaint on dashboard and assigns to an agent
6. Agent logs in, views assigned complaint, updates status and chats with user
7. User tracks complaint status on Status page and communicates via chat
8. Agent marks complaint as Resolved after fixing the issue
9. User sees Resolved status on dashboard

6. Project Setup and Configuration

6.1 Creating Project Folder

10. Create a new folder named 'online complaint registration'
11. Inside that folder create two folders: frontend and backend
12. Open the folder in VS Code

6.2 Frontend Setup (React App)

Open terminal in VS Code and navigate to the frontend folder:

```
cd frontend
```

Create React application:

```
npx create-react-app .
```

Install required libraries:

```
npm install axios react-router-dom @mui/material @emotion/react @emotion/styled bootstrap  
socket.io-client
```

Start the React development server:

```
npm start
```

Frontend runs at: <http://localhost:3000>

6.3 Backend Setup

Open a new terminal and navigate to backend folder:

```
cd backend
```

Initialize Node.js project:

```
npm init -y
```

Install required packages:

```
npm install express mongoose jsonwebtoken bcryptjs dotenv cors multer nodemailer
```

Create the following folder structure:

```
backend/  
  models/      (User.js, Complaint.js, Message.js)  
  controllers/ (authController.js, complaintController.js, adminController.js, messageController.js)  
  routes/      (authRoutes.js, complaintRoutes.js, adminRoutes.js, messageRoutes.js)  
  middleware/  (authMiddleware.js)  
  uploads/     (stores complaint images)  
  .env         (environment variables)  
  server.js    (entry point)
```

6.4 Environment Variables

Create a .env file inside backend folder with:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/complaintdb
JWT_SECRET=complaintsecret123
```

7. Folder Structure

7.1 Frontend (src/)

```
src/
  components/
    admin/
      AccordionAdmin.jsx - Assign agent dropdown
      AdminHome.jsx      - Admin dashboard with stats and tabs
      AgentInfo.jsx      - View and remove agents
      UserInfo.jsx       - View and remove users
    agent/
      AgentHome.jsx      - Agent dashboard with assigned complaints
    common/
      ChatWindow.jsx     - Chat between user and agent
      FooterC.jsx        - Footer component
      Home.jsx           - Landing page
      Login.jsx          - Login form
      SignUp.jsx         - Registration form
    user/
      Complaint.jsx      - Complaint submission form
      HomePage.jsx       - User dashboard
      Status.jsx         - Complaint status tracking
  Images/               - Project images
  App.js                - Main app with routing
  App.css               - Global styles
  index.js              - React entry point
```

7.2 Backend

```
backend/
  controllers/
    authController.js    - register, login functions
    complaintController.js - create, get, update complaints
    adminController.js   - dashboard, assign, manage users
    messageController.js - send and get messages
```

```
middleware/  
  authMiddleware.js    - JWT verification, role checks  
models/  
  User.js              - User schema  
  Complaint.js         - Complaint schema  
  Message.js          - Message schema  
routes/  
  authRoutes.js        - /api/auth endpoints  
  complaintRoutes.js   - /api/complaints endpoints  
  adminRoutes.js       - /api/admin endpoints  
  messageRoutes.js     - /api/messages endpoints  
uploads/  
  .env                 - Environment variables  
server.js              - Express server entry point
```

8. Running the Application

Step 1: Start MongoDB

Open MongoDB Compass and click Connect to connect to mongodb://localhost:27017

Step 2: Start Backend

```
cd backend  
node server.js
```

Expected output:

```
MongoDB connected  
Server running on port 5000
```

Step 3: Start Frontend

```
cd frontend  
npm start
```

Frontend opens automatically at: <http://localhost:3000>

9. API Documentation

All endpoints exposed by the backend with request methods, auth requirements and responses:

Method	Endpoint	Description	Auth	Response
--------	----------	-------------	------	----------

POST	/api/auth/register	Register new user	No	201 - User created
POST	/api/auth/login	Login and get JWT token	No	200 - Token returned
POST	/api/complaints	Submit new complaint	Yes (User)	201 - Complaint saved
GET	/api/complaints/my	Get logged-in user complaints	Yes (User)	200 - Complaints list
GET	/api/complaints/:id	Get complaint by ID	Yes	200 - Complaint detail
PUT	/api/complaints/:id/status	Update complaint status	Yes (Agent)	200 - Status updated
GET	/api/complaints/assigned	Get agent assigned complaints	Yes (Agent)	200 - Complaints list
GET	/api/admin/dashboard	Get all complaints and stats	Yes (Admin)	200 - Dashboard data
PUT	/api/admin/assign	Assign complaint to agent	Yes (Admin)	200 - Complaint updated
GET	/api/admin/agents	Get all agents	Yes (Admin)	200 - Agents list
GET	/api/admin/users	Get all users	Yes (Admin)	200 - Users list
DELETE	/api/admin/users/:id	Delete a user	Yes (Admin)	200 - User removed
DELETE	/api/admin/agents/:id	Delete an agent	Yes (Admin)	200 - Agent removed
GET	/api/messages/:complaintId	Get messages for complaint	Yes	200 - Messages list
POST	/api/messages	Send a message	Yes	201 - Message sent

10. Authentication

The project uses JWT (JSON Web Token) based authentication:

- When a user registers or logs in, the server generates a JWT token signed with the secret key from .env
- The token contains the user ID and role (user / agent / admin)
- The token is stored in the browser localStorage on the frontend
- Every protected API request sends the token in the Authorization header as: Bearer <token>
- The authMiddleware.js verifies the token on every protected route
- adminOnly middleware restricts admin routes to admin role only
- agentOnly middleware restricts agent routes to agent and admin roles
- Passwords are hashed using bcryptjs before storing — plain passwords are never saved in the database

11. Backend Development

The backend is built with Express.js following MVC architecture:

- server.js: Entry point that connects to MongoDB, sets up middleware (CORS, JSON parser, static file serving) and mounts all route files
- authRoutes.js + authController.js: Handles user registration with bcrypt password hashing and login with JWT token generation
- complaintRoutes.js + complaintController.js: Handles complaint CRUD operations with Multer for image uploads
- adminRoutes.js + adminController.js: Admin-only routes for dashboard stats, complaint assignment and user/agent management
- messageRoutes.js + messageController.js: Handles sending and retrieving chat messages between users and agents
- authMiddleware.js: JWT verification middleware with role-based access control functions (protect, adminOnly, agentOnly)

12. Database Development

MongoDB is used as the database with Mongoose ODM for schema modeling:

- User Schema: name (String), email (String, unique), phone (String), password (String, hashed), role (enum: user/agent/admin), timestamps
- Complaint Schema: title, description, category, department, urgency (enum: Low/Medium/High), status (enum: Pending/Assigned/In Progress/Resolved/Rejected), attachment (filename), userId (ref: User), userName, userEmail, agentId (ref: User), timestamps
- Message Schema: complaintId (ref: Complaint), senderId (ref: User), senderRole (enum: user/agent/admin), text, timestamps

The database name is complaintdb running on mongodb://localhost:27017. MongoDB Compass is used to view and manage the collections during development.

13. Frontend Development

The frontend is built with React.js using component-based architecture:

- App.js: Main component with React Router defining all routes (/login, /signup, /home, /complaint, /status, /agent, /admin)

- Home.jsx: Landing page with navbar, hero section and feature cards
- Login.jsx: Login form using Axios POST to /api/auth/login, stores JWT token in localStorage
- SignUp.jsx: Registration form using Axios POST to /api/auth/register
- HomePage.jsx: User dashboard fetching complaints from /api/complaints/my
- Complaint.jsx: Complaint form with file upload sending multipart/form-data to /api/complaints
- Status.jsx: Status tracking page with visual timeline and ChatWindow component
- AgentHome.jsx: Agent dashboard fetching from /api/complaints/assigned with status update buttons
- AdminHome.jsx: Admin dashboard with stats cards, complaints list, AgentInfo and UserInfo tabs
- ChatWindow.jsx: Reusable chat component fetching messages from /api/messages/:complaintId

14. Testing

The application was tested manually covering all major features and user flows:

Test ID	Test Case	Steps	Expected Result	Status
TC-01	User Registration	Fill form with valid name, email, phone, password	Account created, redirected to login	Pass
TC-02	User Login	Login with correct credentials	JWT token generated, redirected to dashboard	Pass
TC-03	Role-based Redirect	Login as admin/agent/user	Admin→/admin, Agent→/agent, User→/home	Pass
TC-04	Complaint Submission	Fill complaint form with image	Complaint saved in MongoDB with Pending status	Pass
TC-05	Status Tracking	Click Track on a complaint	Visual timeline shows correct current status	Pass
TC-06	Admin Assignment	Admin assigns agent to complaint	Status changes to Assigned, agent notified	Pass
TC-07	Agent Status Update	Agent updates status to Resolved	Status reflected on user dashboard	Pass
TC-08	Chat Messaging	User sends message to agent	Agent receives message in chat window	Pass
TC-09	Invalid Login	Login with wrong password	Error message shown, no token generated	Pass

TC-10	MongoDB Verification	Check collections in Compass	users, complaints, messages data saved correctly	Pass
-------	-----------------------------	------------------------------	--	-------------

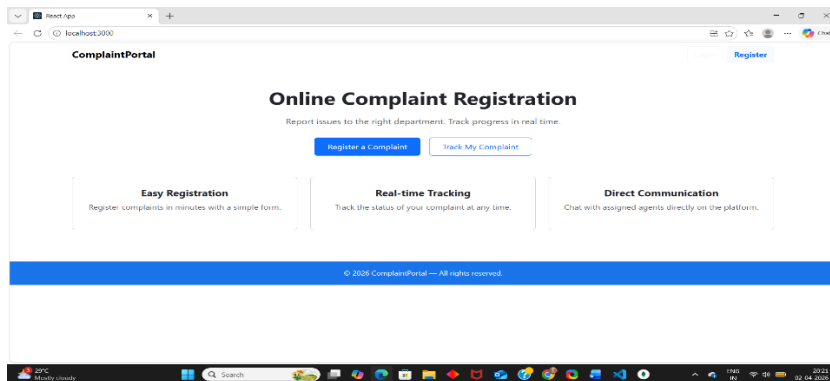
15. Known Issues

- Real-time chat uses REST API polling instead of Socket.io WebSockets
- No forgot password or OTP verification feature implemented
- Agent accounts must be manually assigned role through MongoDB Compass
- Images are stored locally in uploads folder — cloud storage not integrated

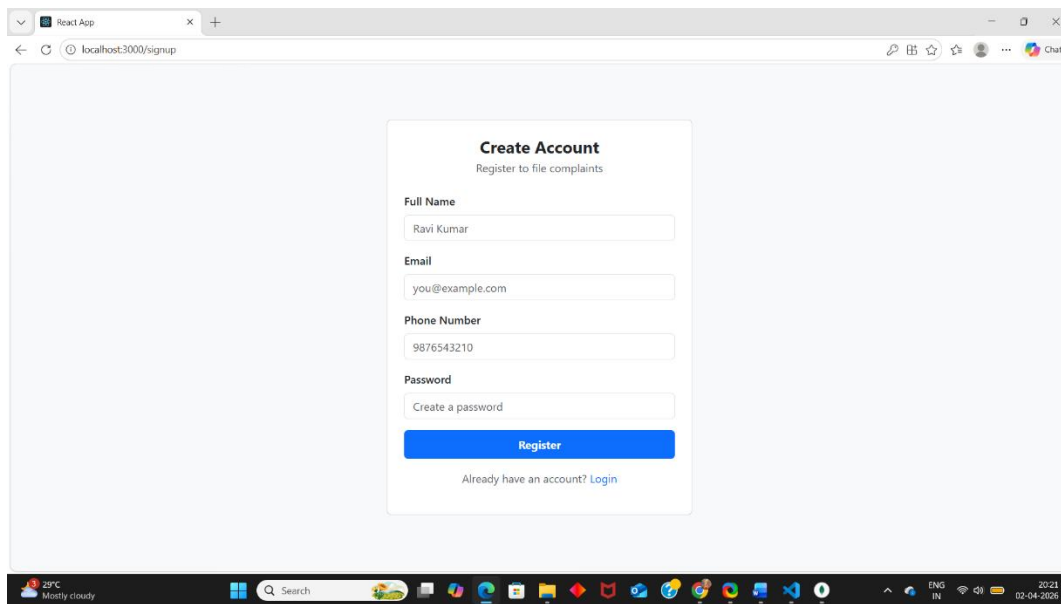
16. Future Enhancements

- Add Socket.io for real-time live chat and instant notifications
- Implement forgot password with OTP verification via email
- Add Google OAuth social login
- Integrate cloud storage (AWS S3 or Cloudinary) for complaint attachments
- Build mobile app version using React Native
- Add analytics dashboard with complaint trend charts
- Implement automated email notifications using Nodemailer
- Add AI-based complaint category auto-detection
- Multi-language support for regional users
- Complaint priority escalation for overdue complaints

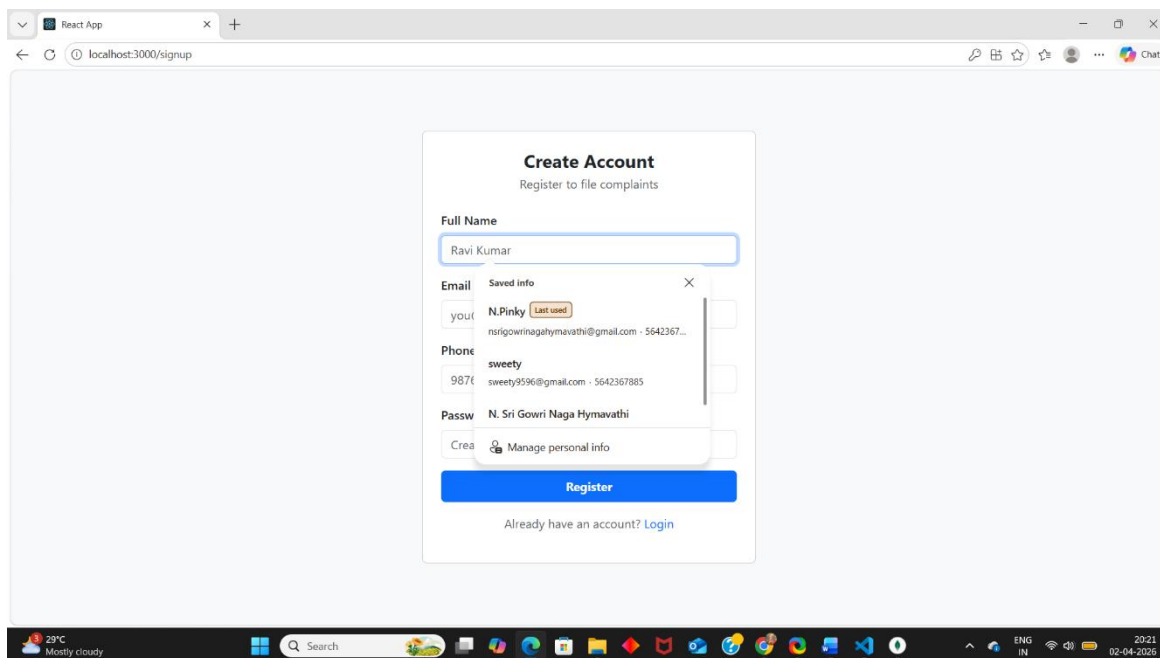
17. Screenshots / Demo



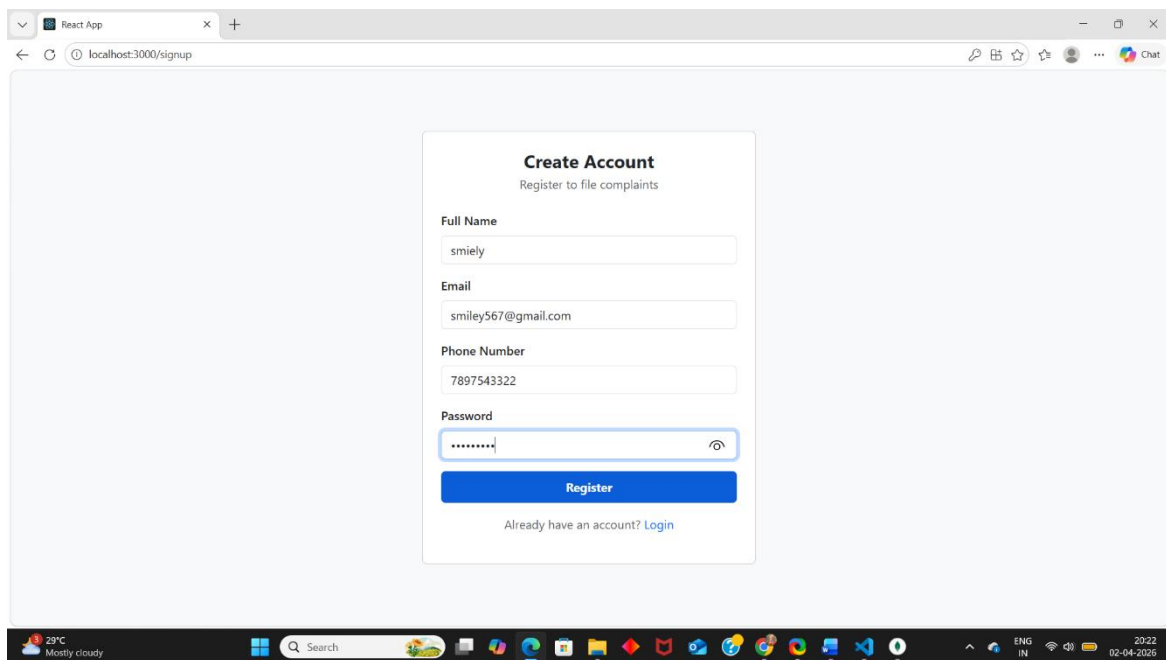
This is my initial page



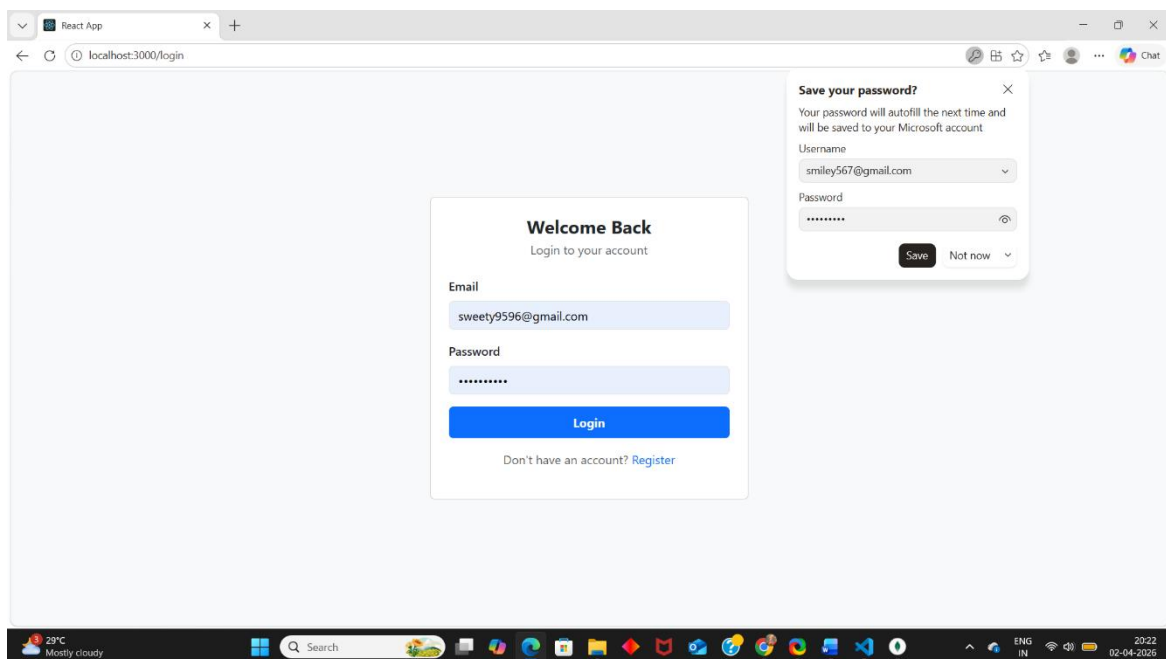
Here you can create an account or login

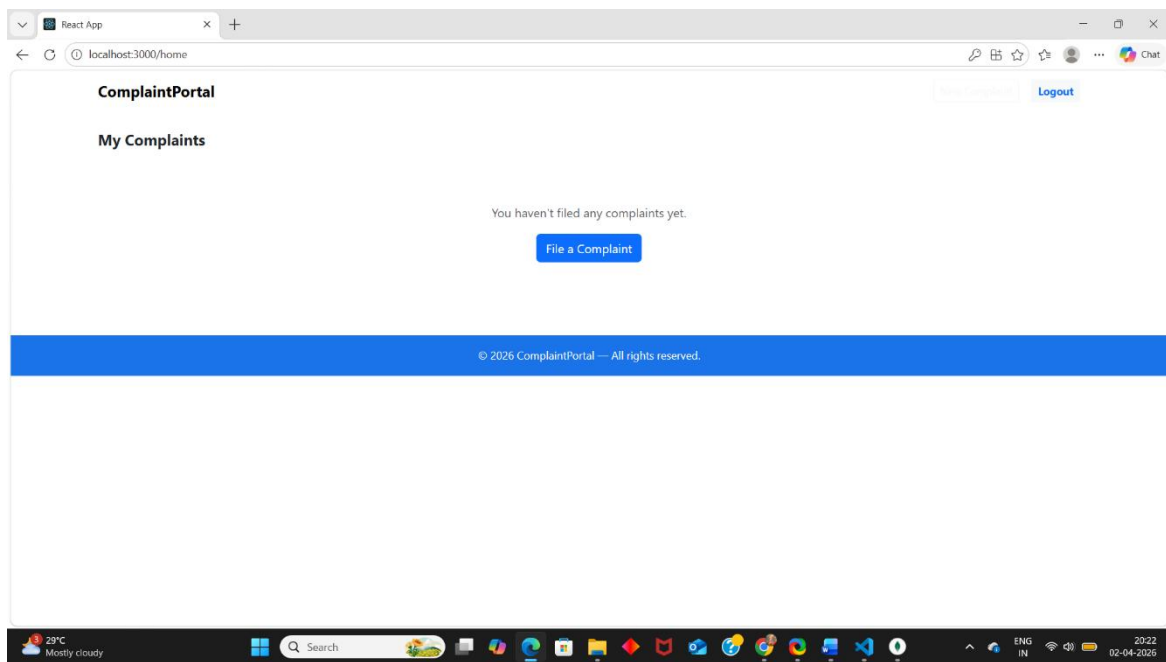


Here it is showing previous registrations

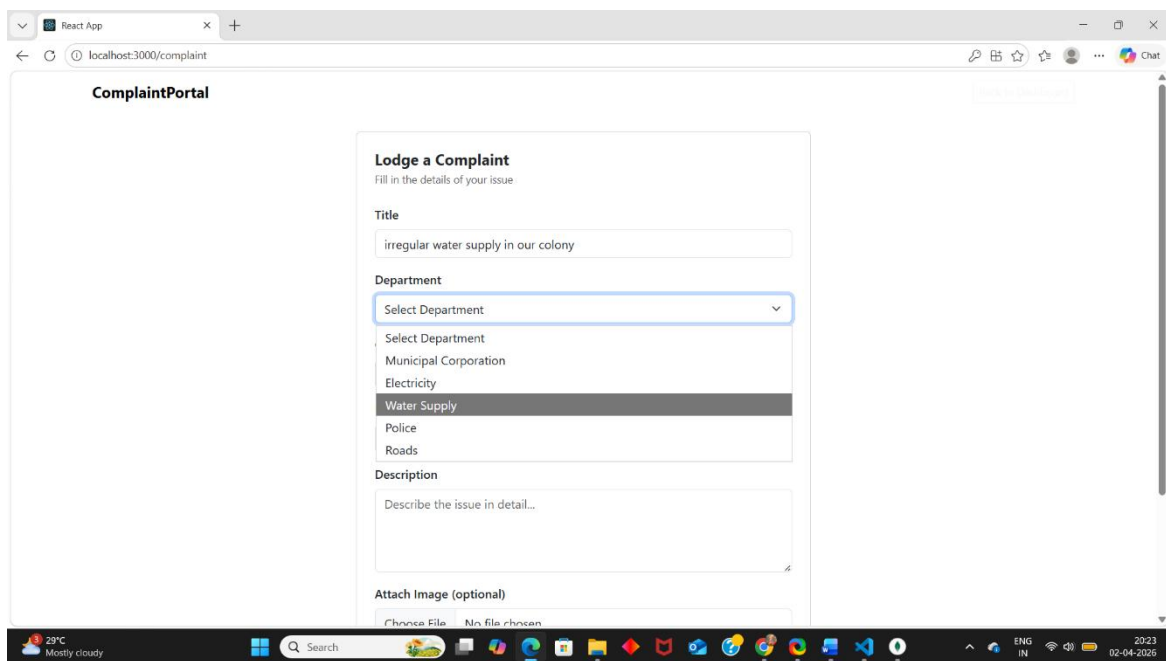


I have created new account

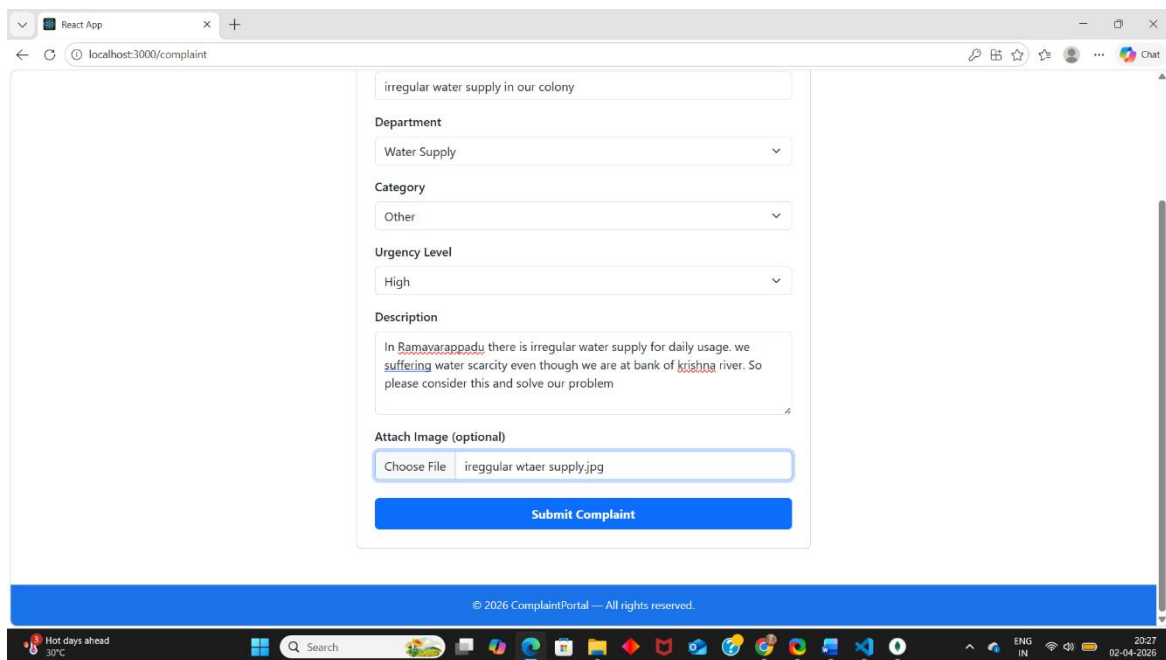




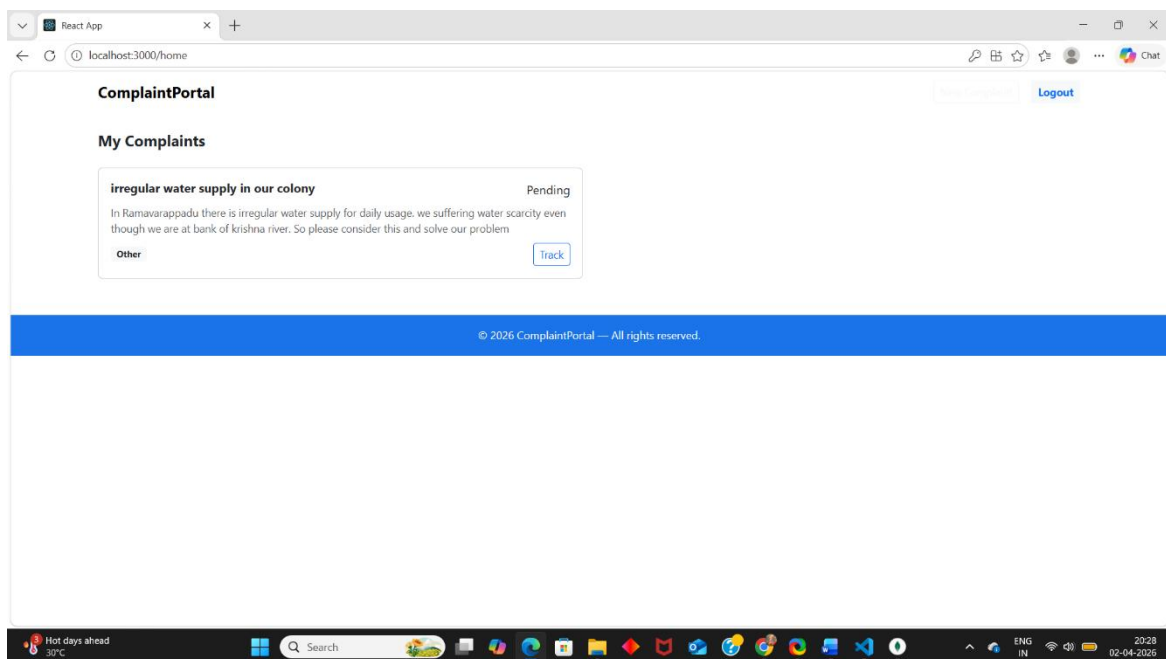
This is my home page



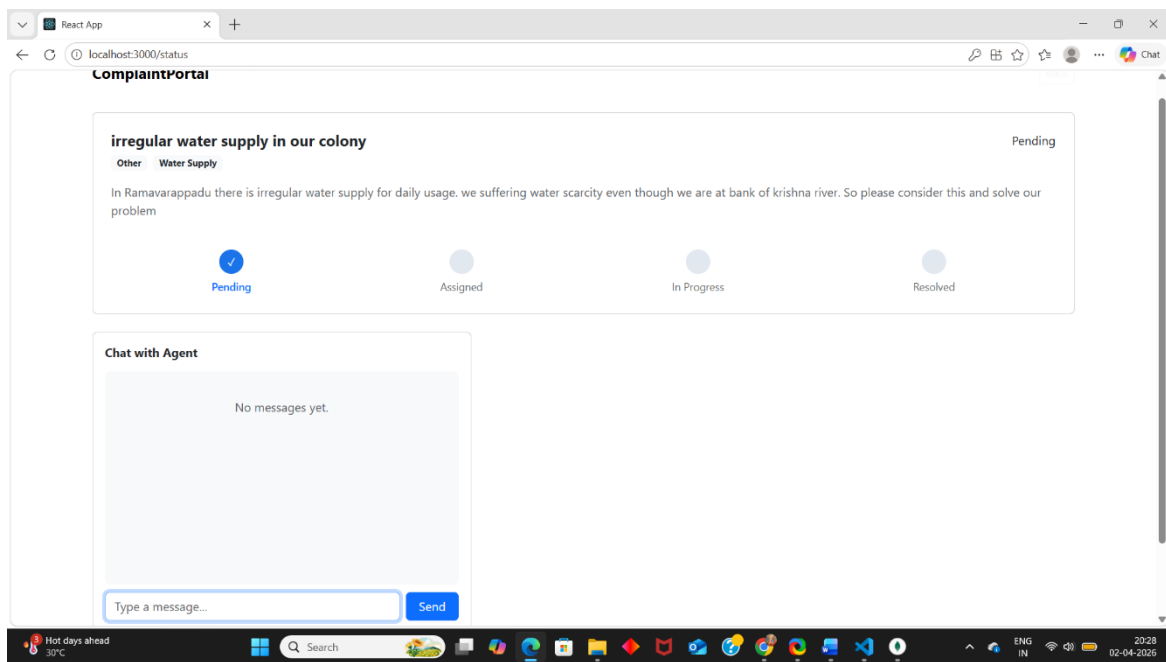
Here I am giving a complaint



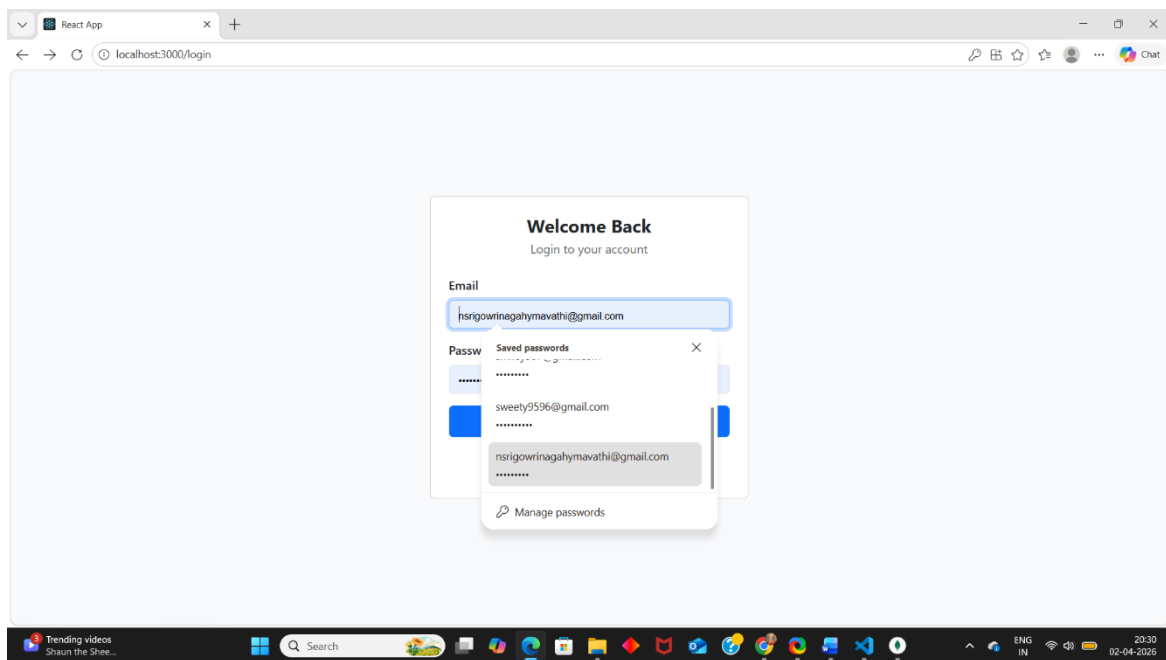
I completed giving complaint and clicked on submit



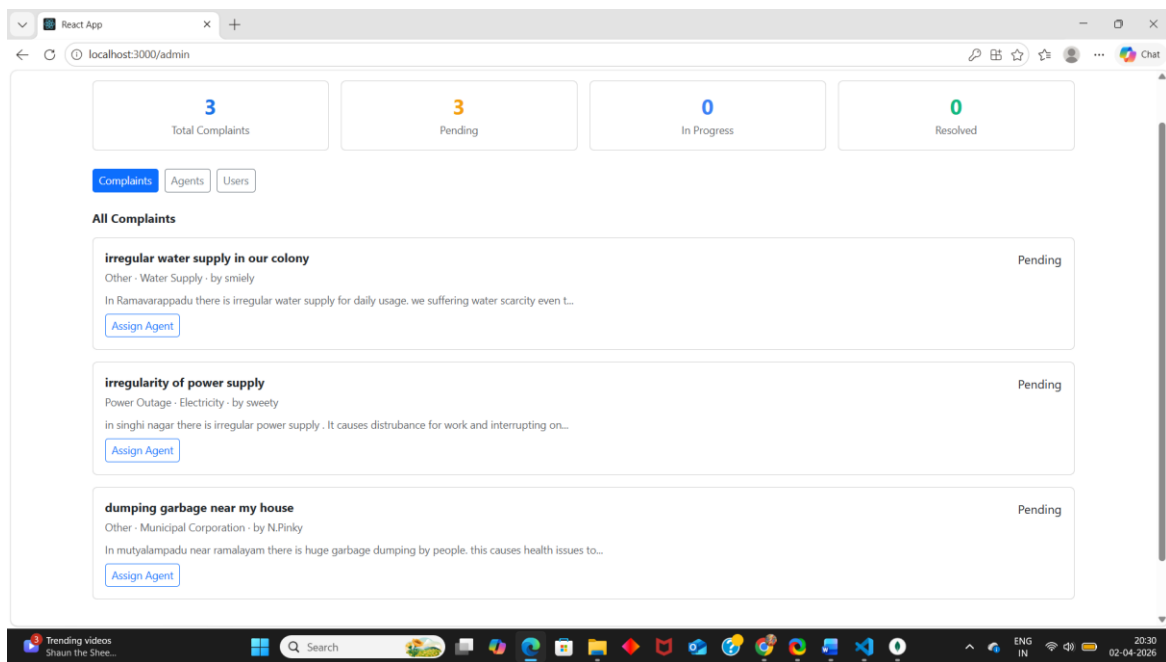
My complaint is updated here



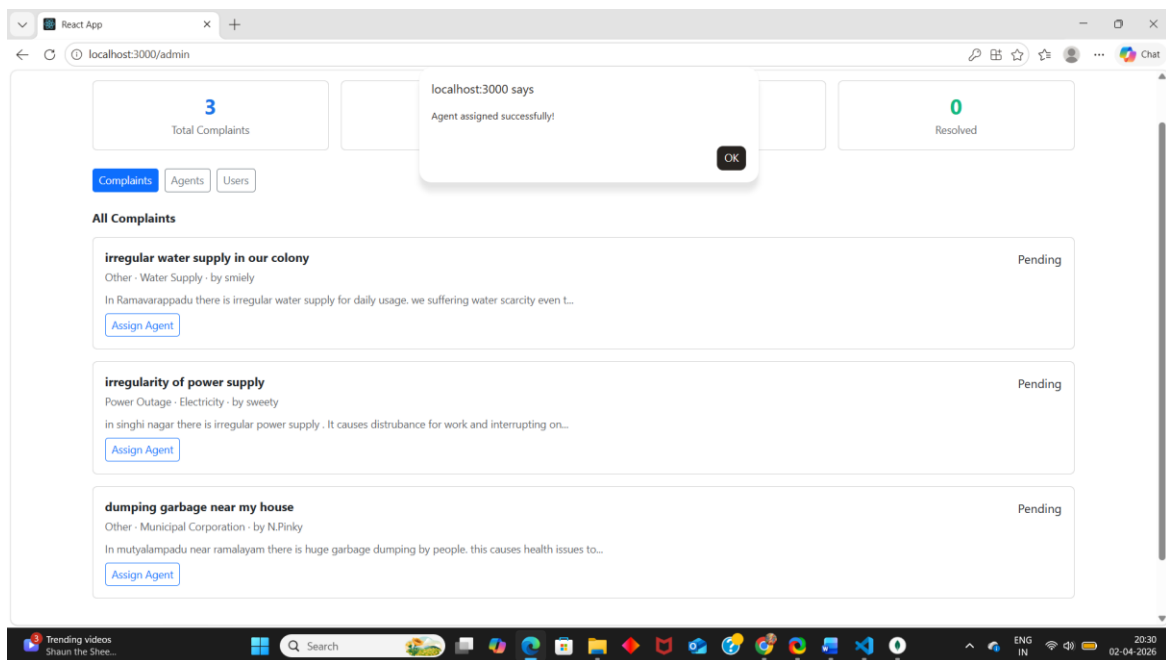
This is showing my progress in status bar



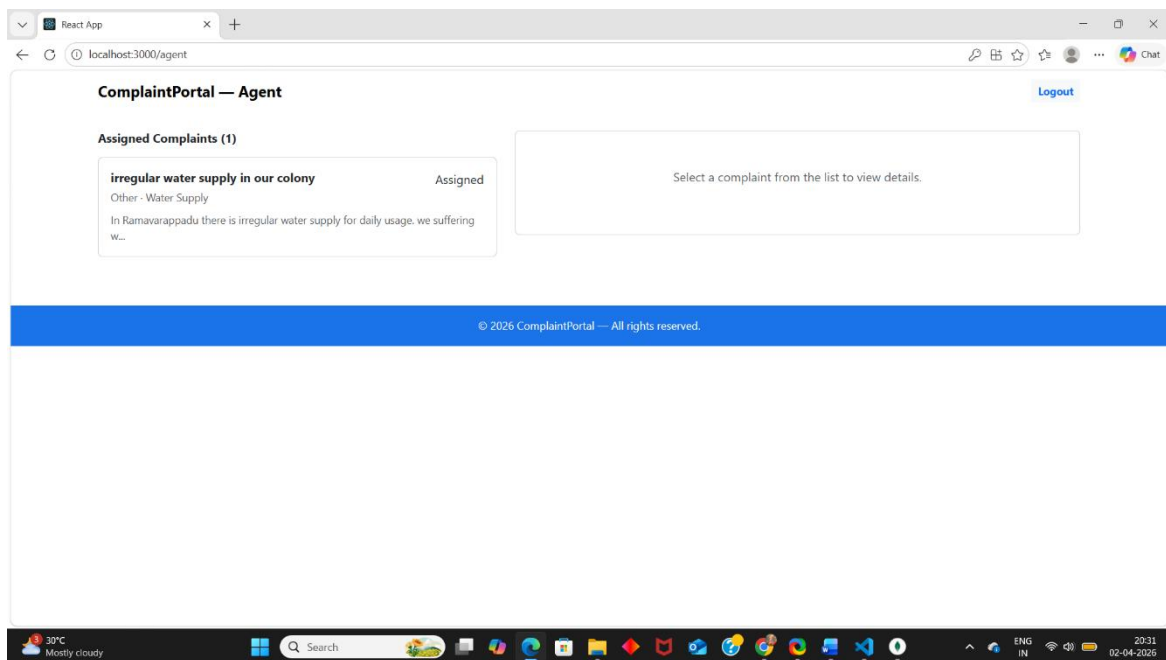
Now I am logging in into account that I have created before



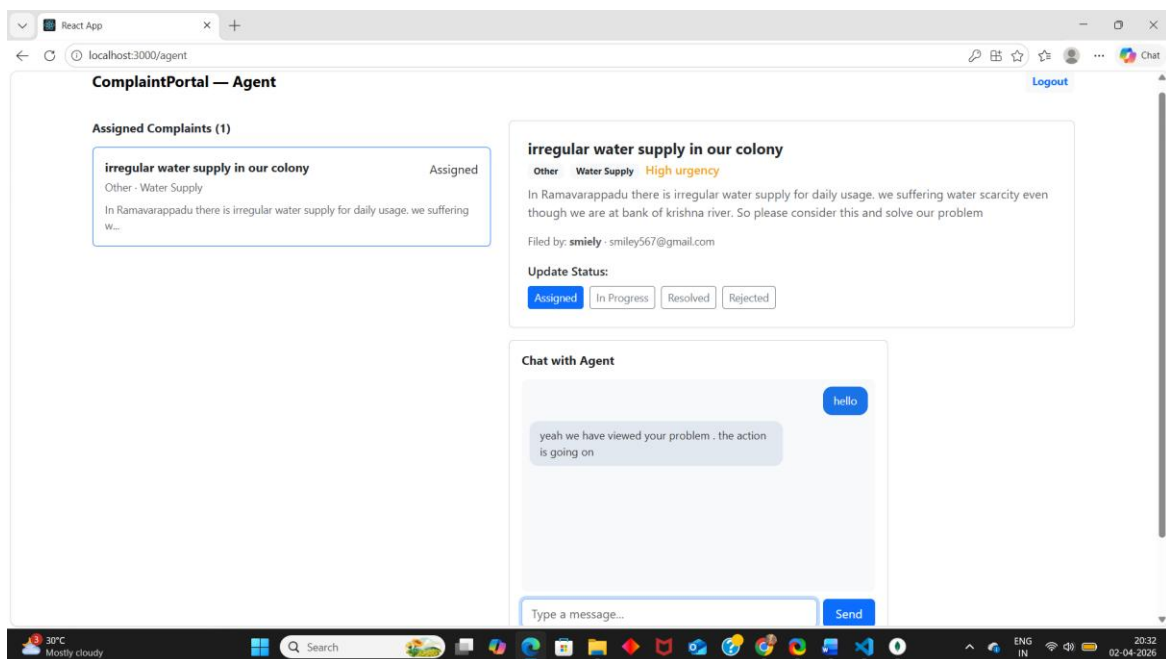
Here I am seeing all complaints by users in admin bar



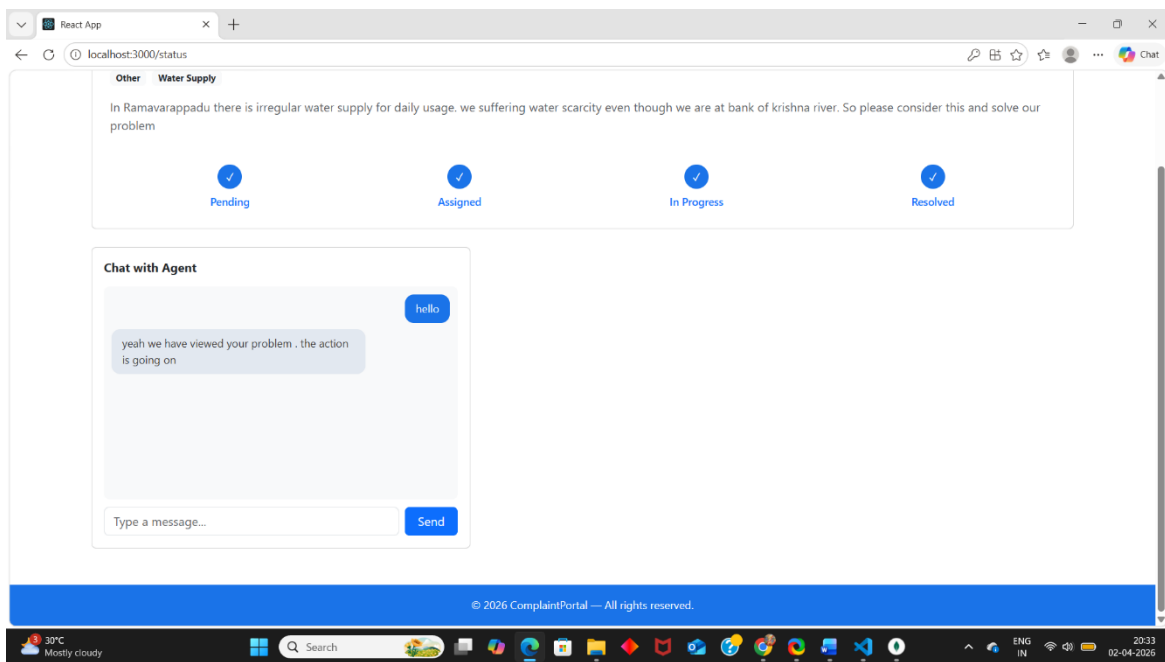
Here I assigned an agent to user to resolve problem



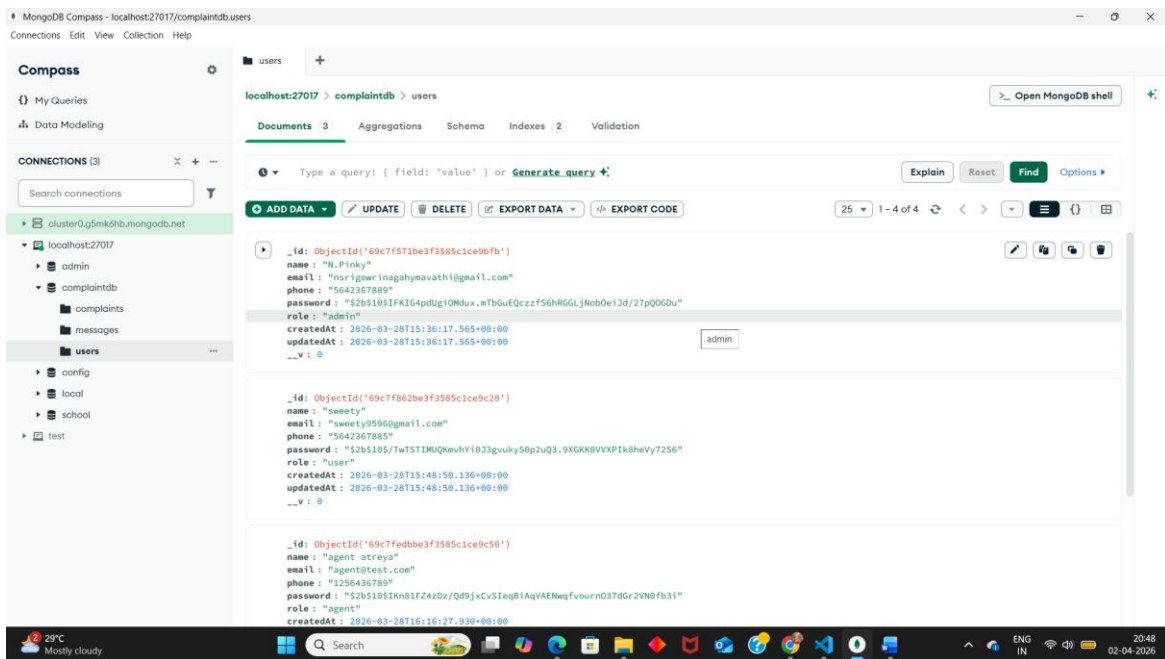
This is agent account and he can see the assigned complaint



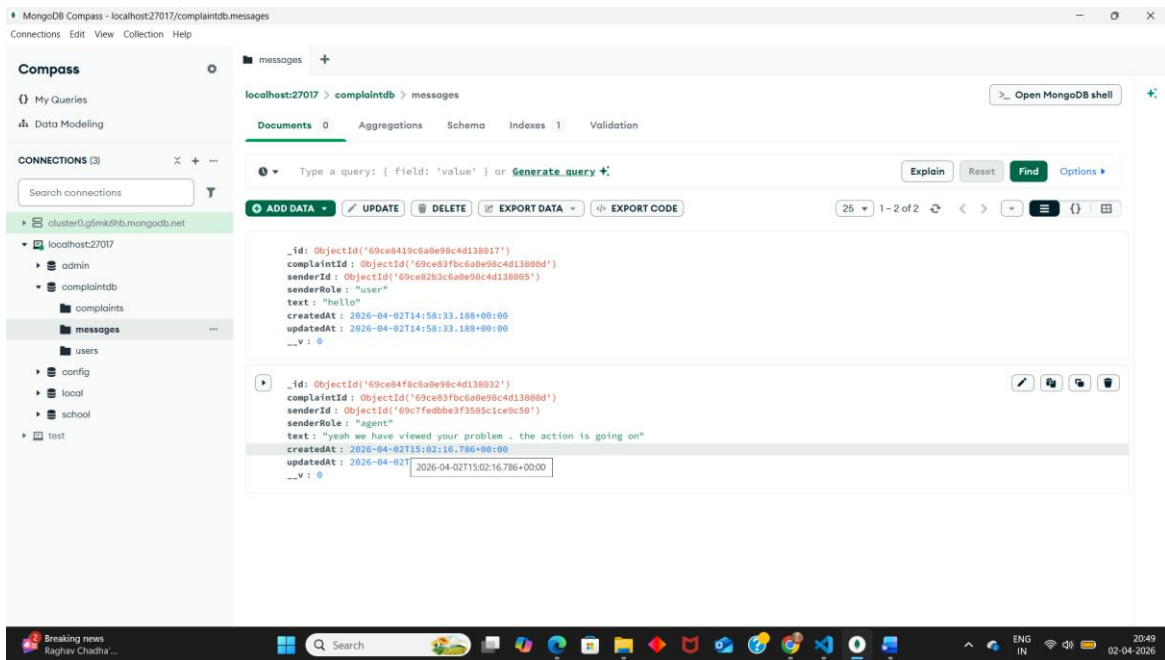
There is chat box so that agent can easily interact with the users



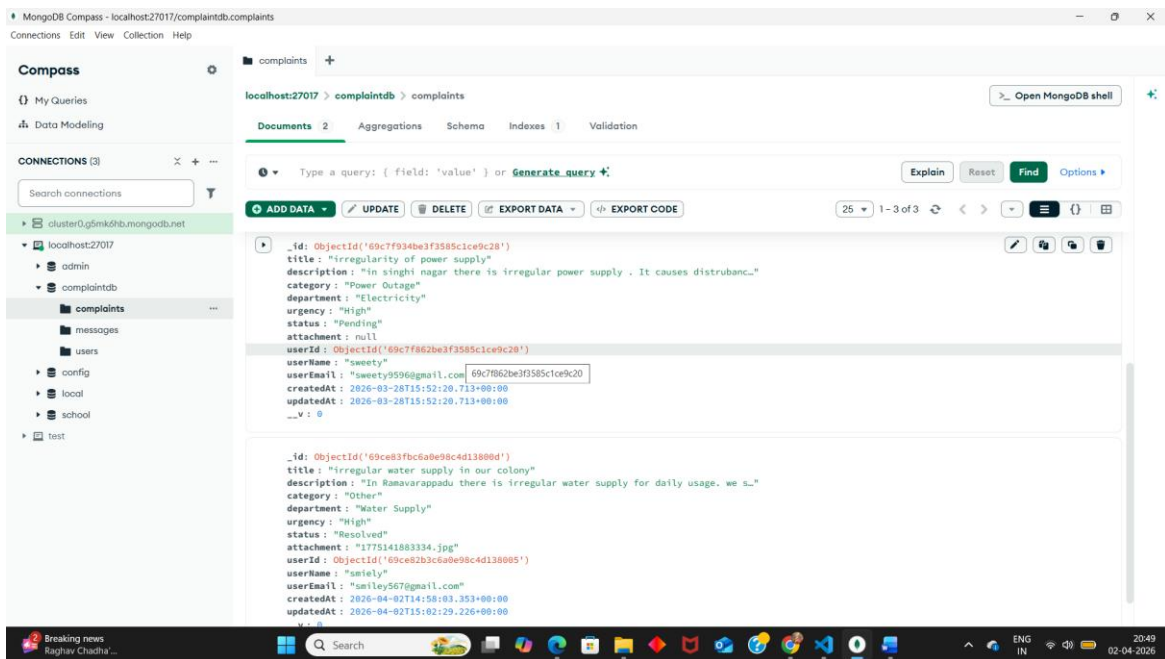
After resolving it will be updated like this



This is mongodb database . This is showing all users



Here it is showing chat between user and agent



The information is stored securely and showing status of the complaints